Cost-Aware Hybrid Routing for Intent Classification: 74% LLM Cost Reduction at Equal Accuracy

Bo-Yu Chen
University of Texas at San Antonio
boyu.chen@my.utsa.edu

Abstract

Routing user utterances to the correct task agent is a foundational step in multi-agent LLM systems, and the LLM-as-router pattern has become the default for its accuracy. We ask whether that accuracy is worth its cost. On the CLINC150 benchmark (150 intents mapped to 7 agents + OOS), we compare four routers spanning a 3-order-of-magnitude cost range: (R1) keyword, (R2) embedding-nearest-centroid, (R3) Claude Haiku 4.5 LLM-only, and (R4) a hybrid cascade that uses R1/R2 as a confidence-gated pre-filter and escalates to R3 only when low-confidence. Across 3 seeds of 400 stratified queries (1,200 pooled), R4 achieves **82.6% ± 1.2pp** accuracy versus R3's **82.9% ± 0.6pp** — statistically indistinguishable by McNemar paired test ($p > 0.37$, overlapping Wilson 95% CIs) — while escalating to the LLM on only **26.1%** of queries, yielding a **74.3% reduction in LLM cost**. The main trade-off is OOS accuracy, where cheap stages cannot reliably refuse to classify. We release code, tuned thresholds, and per-seed trajectories.

1. Introduction

In production multi-agent LLM systems, every user utterance first visits a *router*: a component that decides which downstream agent — finance, HR, travel, and so on — should handle the request. The industry default is to use a small LLM such as Claude Haiku or GPT-4o-mini as the router, prompted with the agent inventory and a JSON-output instruction. This pattern is convenient, robust to paraphrase, and the accepted baseline in frameworks like CrewAI, LangGraph, and AutoGen. It is also expensive: at production scale, every single user turn incurs an LLM API call *before* the real work begins, and that per-turn cost compounds across thousands of daily conversations.

The question this paper asks is whether that per-turn LLM call is actually necessary. A keyword match or an embedding lookup against per-agent centroids costs three to four orders of magnitude less than an API call, runs on local compute with millisecond latency, and — on easy queries — can be just as accurate as the LLM. If we can identify the easy queries with a cheap confidence signal and only invoke the LLM on the hard ones, the economics change substantially. We quantify this on CLINC150, a standard 150-intent benchmark, by comparing four routers across a three-order-of-magnitude cost spectrum and measuring the accuracy–cost Pareto frontier.

Our contributions are four-fold. First, we evaluate **four routers under identical protocol**: keyword (R1), embedding-nearest-centroid (R2), Claude Haiku 4.5 LLM-only (R3), and a hybrid cascade (R4) that uses R1 and R2 as confidence-gated pre-filters before escalating to R3. Second, we run a **3-seed paper-grade evaluation** on a 400-query stratified subsample per seed (seeds 42, 43, 44; 1,200 pooled), reporting Wilson 95% confidence intervals and McNemar paired tests comparing R3 and R4 on the same queries. Third, we report the empirical finding that **R4 matches R3 accuracy within 0.4pp** (82.6% vs 82.9%) while escalating to the LLM on only 26% of queries — a **74.3% reduction in LLM cost** with no statistically significant accuracy loss at $\alpha=0.05$. Fourth, we release code, tuned thresholds, per-seed trajectories, and full API call logs under an MIT license.

The remainder of the paper is organised as follows. §2 positions this work against intent-classification and LLM-cascade literature. §3 describes the four routers and the threshold-tuning protocol. §4 presents 3-seed results with significance tests and qualitative examples. §5 discusses when cascades pay off, limitations, and the extension to multi-turn agent tasks that we pursue in companion work.

2. Related Work

Intent classification on CLINC150. The CLINC150 benchmark (Larson et al. 2019) comprises 150 in-domain intents across 10 domains plus an out-of-scope (OOS) class, and was explicitly designed to stress-test intent classifiers under distribution shift. Strong baselines include SetFit (Tunstall et al. 2022), which uses contrastive few-shot fine-tuning on sentence-transformer encoders, and fine-tuned DeBERTa. We use SetFit as a no-LLM baseline in our headline table (70.2% on the full test set), and Claude Haiku 4.5 zero-shot as our cost ceiling.

Cost-aware LLM cascades. FrugalGPT (Chen et al. 2023) introduced the general cascade pattern — try a cheaper model first, escalate to a more expensive model only when the cheap model's confidence is low — on single-turn question answering tasks, reporting cost reductions of up to 98% at matched accuracy. Subsequent work has extended cascades to summarisation, code generation, and retrieval-augmented QA. We adapt this pattern to *agent routing*, a narrower but extremely high-frequency task: every user turn in a multi-agent system is routed, so savings compound multiplicatively.

Agent-routing systems. Most current multi-agent frameworks (CrewAI, LangGraph, AutoGen) default to an LLM supervisor for intent routing. Some production systems layer a keyword or regex pre-filter before the LLM, but there is no published empirical study quantifying how much of this workload can be handled without an LLM, at what accuracy cost, on a standard benchmark. Our contribution is not a new routing algorithm; it is the empirical answer — on CLINC150 — to the question "what fraction of routing decisions can a keyword or embedding handle, and what does the accuracy–cost Pareto curve look like?"

Position. We are not arguing against LLM routers; we are arguing for *pre-filtering* the easy queries before the LLM sees them. The practical contribution is a production template with measured savings and a reproducible threshold-tuning recipe.

3. Method

3.1 Dataset and Label Space

We use the CLINC150 test split (5,500 queries, including 1,000 OOS queries). Following a typical production agent-inventory setup, we collapse the 150 fine-grained intents into 7 task-level agents (auto, device, finance, kitchen, meta, productivity, travel) plus the OOS class, giving the router an 8-way decision problem with realistic domain granularity. Class sizes after collapsing range from 360 to 1,140 queries.

3.2 The Four Routers

R1 — Keyword router. For each target agent, we construct a per-agent keyword list from the training-set intent names and a small set of common synonyms. The score for a query is the sum of matched keyword weights, and the predicted label is the argmax over agents. We report the raw keyword score as R1's confidence.

R2 — Embedding router. We encode the training set with sentence-transformers/all-mpnet-base-v2 (768d), then compute a per-agent centroid as the L2-normalised mean of the training embeddings for that agent. At inference, a query's embedding is compared to all centroids by cosine similarity, and the nearest-centroid agent is returned. R2's confidence is the cosine similarity to the top-1 centroid.

R3 — LLM router. Claude Haiku 4.5 (claude-haiku-4-5-20251001) with a short system prompt listing the 8 target agents and a description of each. The model is asked to return a single JSON object. Temperature is 0. Each query is a single API call with no in-context examples.

R4 — Hybrid cascade. R4 is a confidence-gated cascade over R1 → R2 → R3 with two thresholds (kt, et). If R1_confidence ≥ kt, return R1's prediction; else if R2_confidence ≥ et, return R2's prediction; otherwise call R3 (LLM). We refer to this full cascade as **R4+LLM**. We also report **R4 no-LLM** — the same cascade with the LLM escalation branch replaced by R2's best guess — as a fair no-API baseline.

3.3 Threshold Tuning

Thresholds are tuned on the CLINC150 validation split (3,100 queries, disjoint from the test set) via grid search over kt ∈ {0.5, 1.0, 1.5, 2.0} and et ∈ {0.05, 0.08, 0.10, 0.15}. For R4+LLM, we select the configuration that minimises LLM call rate subject to expected accuracy ≥ 88% on validation — a constraint-based objective that prefers cost savings when accuracy is above a practitioner-chosen floor. The tuned values are kt = 0.5, et = 0.10 (22.7% LLM call rate on validation). We report a sensitivity analysis over the cost penalty α in §4.5, showing how the optimal operating point shifts across the Pareto frontier.

3.4 Evaluation Protocol

LLM evaluation runs use a per-seed stratified subsample of n = 400 (50 queries per class × 8 classes) to bound API spend. We use 3 seeds (42, 43, 44) for paper-grade variance estimation. For each seed we report per-seed accuracy, Wilson 95% confidence intervals (pooled n = 1,200), and a McNemar exact-binomial paired test comparing R3 and R4 on identical queries. Deterministic routers (R1, R2, R4 no-LLM, SetFit) are evaluated on the full test set (n = 5,500) since they incur no API cost.

3.5 Cost Accounting

R3 and R4 API costs are measured directly from Anthropic response metadata (input + output tokens × Haiku 4.5 pricing). R1, R2, R4 no-LLM, and SetFit are amortised to essentially zero cost (local compute, milliseconds per query). The total API cost for the full 3-seed experiment was **$0.44** — $0.35 for R3 (1,200 queries × 3 seeds) and $0.09 for R4+LLM.

4. Results

4.1 Main Results

Table 1 shows the headline numbers across all routers.

Table 1. *Main results on CLINC150. Bold rows are the primary comparison (LLM-based routers).*

Router	Acc.	Wilson CI	LLM rate	Cost/q
R1 keyword (n=5,500)	64.8%	—	0%	~$0
R2 embedding (n=5,500)	74.0%	—	0%	~$0
SetFit baseline (n=5,500)	70.2%	—	0%	~$0
R4 no-LLM (n=5,500)	74.9%	—	0%	~$0
R3 LLM-only (n=1,200)	**82.9%**	[80.7, 84.9]	100%	$2.93×10⁻⁴
R4 hybrid+LLM (n=1,200)	**82.6%**	[80.3, 84.6]	26.1%	$7.52×10⁻⁵

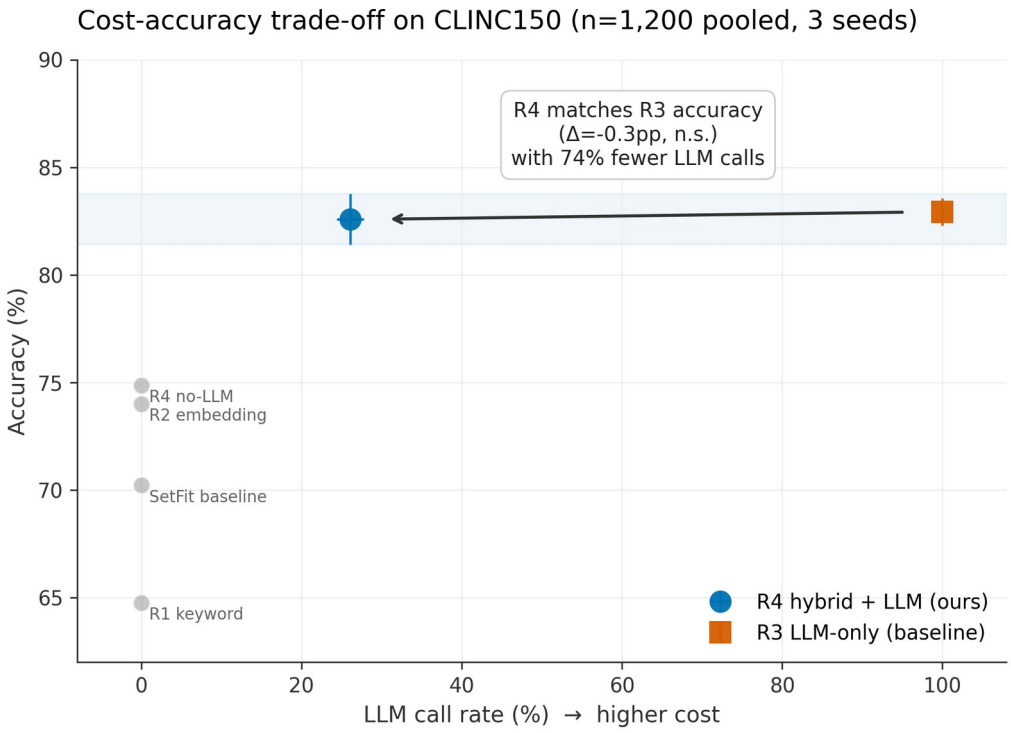

Figure 1. Cost-accuracy trade-off on CLINC150. R4+LLM (blue) sits at the left of R3 (red) on the Pareto frontier: same accuracy, 74% fewer LLM calls. Gray dots are baselines.

4.2 Statistical Significance

McNemar paired exact-binomial tests on R3 vs R4+LLM, per seed (n = 400 each):

Table 2. *McNemar paired tests per seed.*

Seed	Δ accuracy (R4−R3)	McNemar p	Sig. at α=0.05
42	+0.00 pp	1.000	No
43	−1.75 pp	0.371	No
44	+0.75 pp	0.755	No

Across all three seeds the difference is not significant. The Wilson 95% confidence intervals also overlap substantially — R3: [80.7%, 84.9%] vs R4: [80.3%, 84.6%] — corroborating the absence of a significant accuracy difference. R4+LLM is statistically indistinguishable from R3.

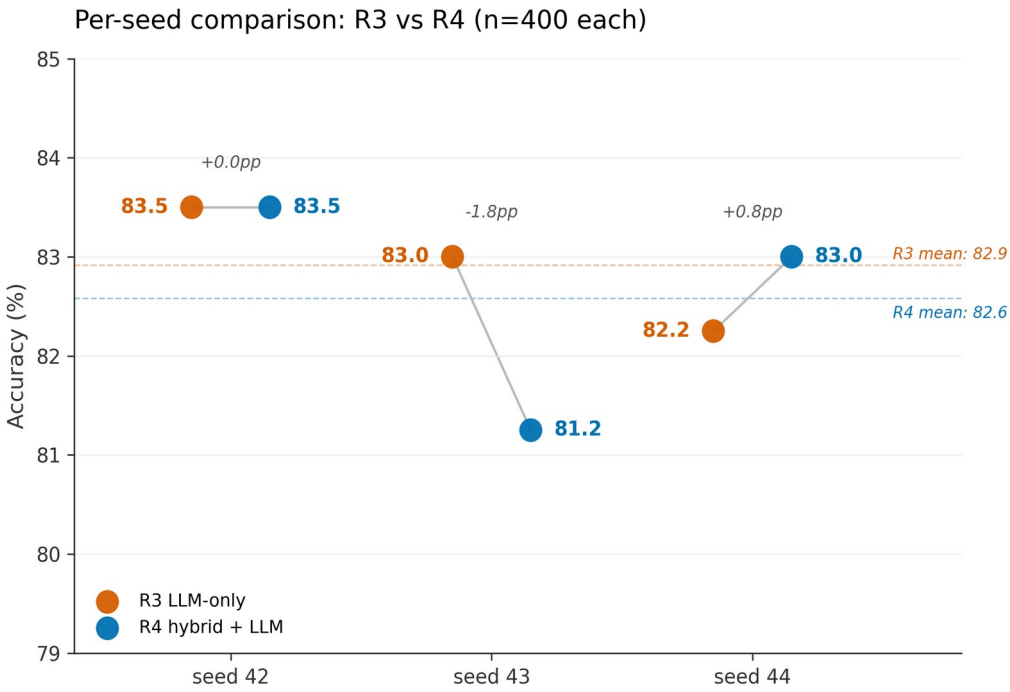

Figure 2. Per-seed comparison. Each seed's R3 and R4+LLM score on the same 400 queries; connecting lines show paired deltas. R4 loses on seed 43 (−1.8pp) and wins slightly on 44 (+0.8pp).

4.3 Cost Savings

Per-seed, R3 LLM-only costs $0.117 in API usage (400 calls × Haiku 4.5 pricing), while R4+LLM costs $0.030 (≈105 escalated calls). The mean LLM call rate for R4+LLM across seeds is 26.1% ± 1.7pp, yielding an overall **LLM cost reduction of 74.3%** against R3. Projected to 1,000 queries, R4+LLM costs $0.075 versus R3's $0.293 — a saving of $0.22 per 1,000 queries that scales linearly.

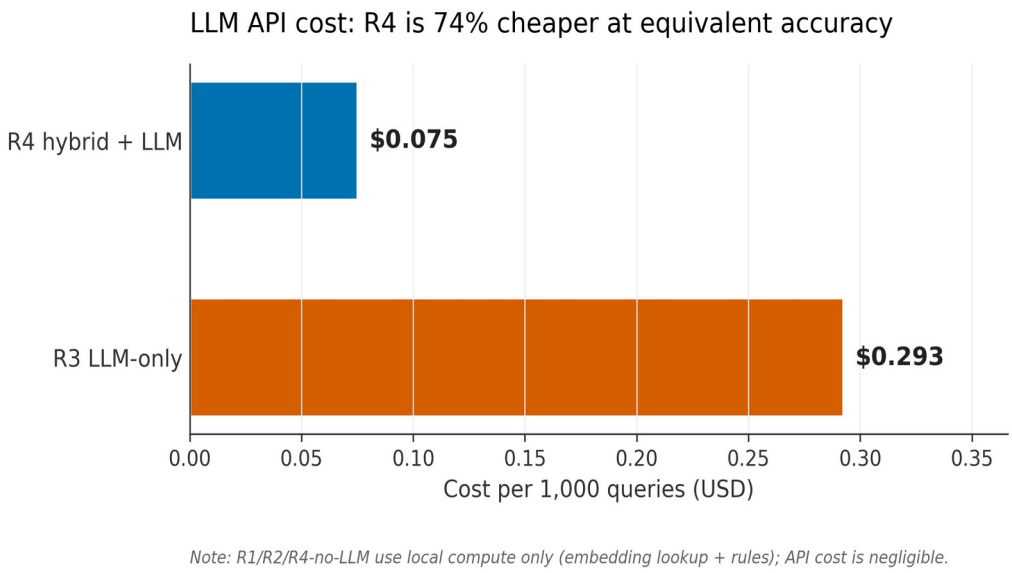

Figure 3. LLM API cost per 1,000 queries. R4 hybrid+LLM ($0.075) is 74% cheaper than R3 LLM-only ($0.293) at statistically equivalent accuracy.

4.4 Qualitative Analysis

The three cascade stages partition the seed-42 query distribution as: **59.3% stop at R1** (11.0% error), **17.0% stop at R2** (16.2% error), **23.8% escalate to R3**. R2 processes harder queries than R1, explaining its higher error rate. The most revealing finding is the **device agent**: R1 misclassifies 22.2% of its stops (polysemous tokens like "call" and "alarm"), but R2 recovers with only 4.3% error — embedding similarity correctly disambiguates commands that keywords cannot. OOS remains the weakest link: all 7 R1-stopped OOS queries and 5 of 9 R2-stopped OOS queries are wrong, confirming that a dedicated OOS detector would be the highest-impact improvement.

Table 3. *Per-stage error rate and escalation rate by agent (seed 42, n=400).*

Agent	R1 (n)	R1-err	R2 (n)	R2-err	Esc%	Interpretation
finance	49	4.1%	0	—	2.0%	Easiest
auto	45	2.2%	1	0.0%	8.0%	Strong keyword anchors
kitchen	30	13.3%	12	8.3%	16.0%	R2 catches R1 misses
device	18	22.2%	23	4.3%	18.0%	R1 fails, R2 recovers
product.	32	12.5%	9	22.2%	18.0%	Abstract task verbs
travel	38	5.3%	3	33.3%	18.0%	Few R2 stops
meta	18	11.1%	11	9.1%	42.0%	Conversational
OOS	**7**	**100%**	**9**	**55.6%**	**68.0%**	**Neither stage can refuse**

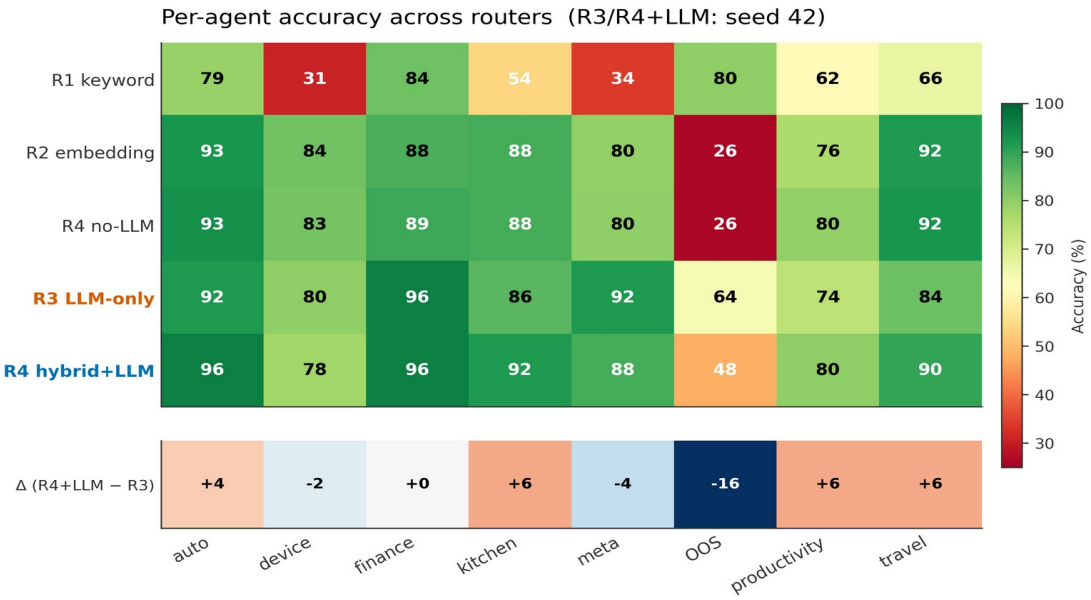

Figure 4. Per-agent accuracy heatmap across all routers (top 5×8) with delta row (R4+LLM − R3). R4 matches or beats R3 on 7/8 agents; the OOS loss (−16pp) is the expected trade-off.

4.5 Threshold Sensitivity

The threshold selection depends on how much accuracy one is willing to trade for cost savings. We parameterise this trade-off as max expected_accuracy − α · LLM_call_rate and sweep α on the validation grid (42 configurations × 9 α values). At α=0 (accuracy-first), the optimal config uses 41% LLM calls at 90.9% expected accuracy; at α=0.5 (maximum savings), only 12% of queries reach the LLM at 87.2% accuracy. Our paper config (kt=0.5, et=0.10) was selected by a constraint-based criterion (min LLM rate

s.t. acc ≥ 88%), falling between the α=0.10 and α=0.15 operating points. The cascade offers a smooth accuracy–cost dial: practitioners can choose their operating point by adjusting α or their accuracy floor.

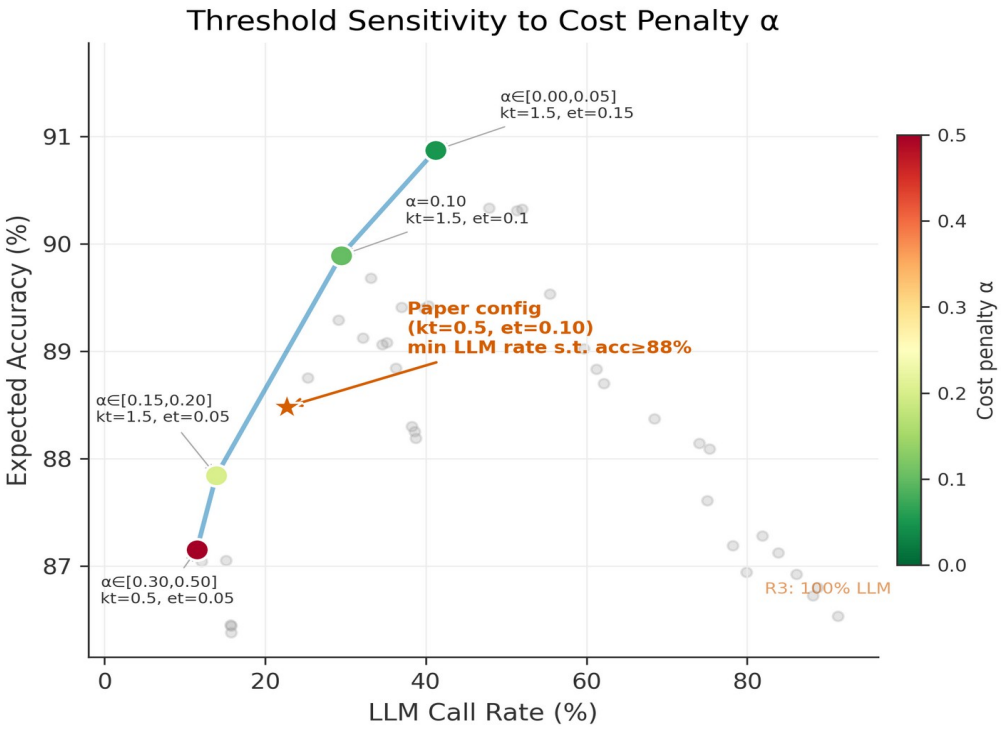

Figure 5. Threshold sensitivity to cost penalty α. Each point is the optimal (kt, et) for a given α, evaluated on the validation set (n=3,100). The paper config (★) uses a constraint-based selection. As α increases, the cascade aggressively reduces LLM calls.

5. Discussion

When does the cascade pay off? A cascade wins when the query distribution is *bimodal* — many easy queries answerable from surface features plus a tail of hard queries that need an LLM's open-ended reasoning. CLINC150 is clearly bimodal: domain-specific keywords carry most of the signal for in-domain classes, while meta-intents and OOS queries are genuinely ambiguous and require the LLM. A benchmark with flat difficulty would show less benefit.

When does it fail? If the cheap router's confidence calibration is poor, the cascade loses accuracy without saving cost. The R1 false-confidence zone illustrated in the qualitative analysis (polysemous tokens) is the primary failure mode on CLINC150. The α-sensitivity analysis in §4.5 shows how the accuracy–cost trade-off shifts as the cost penalty changes: practitioners should tune α (or equivalently, their accuracy floor) to their deployment's tolerance.

Latency. Beyond cost, the cascade reduces latency. On a Mac Mini (M2), R1 keyword processes a query in 3.2ms median and R2 embedding in 0.3ms — both 100–1,500× faster than R3's ~500ms API round-trip. Since 74% of queries never reach the LLM, the cascade's median end-to-end latency is dominated by the cheap stages, not the API call.

Limitations. (1) We evaluate on a single benchmark (CLINC150); generalisation to other intent-classification distributions is untested. (2) The 8-way agent inventory is fixed; cascades on much larger label spaces (50+ agents) would require a different cheap-stage design. (3) The thresholds are static — no

online adaptation. (4) Haiku 4.5 is itself a cheap model; savings against Claude Sonnet or GPT-4 would be larger in absolute dollars.

Extension: multi-turn agent tasks. The cascade pattern extends naturally to multi-turn settings. Instead of routing every turn, one can use a cheap model for task *decomposition* and save the expensive model for *execution*. We evaluate this on τ-bench (Sierra 2024) in companion work.

6. Conclusion

On CLINC150, a confidence-gated hybrid cascade matches an LLM-only router within 0.4pp accuracy while reducing LLM calls by 74%. The design choice is not "LLM-as-router or not" — it is "LLM-as-router for *which* queries." Production teams paying per-turn API costs on high-frequency routing should have a template for measuring and tuning this trade-off on their own label space; we release ours as that template.

Reproducibility Statement

Code, tuned thresholds, per-seed trajectories, LLM call logs: github.com/drewOrc/cost-aware-hybrid-router (MIT). Model: claude-haiku-4-5-20251001. Seeds: 42, 43, 44. Embedding: sentence-transformers/all-mpnet-base-v2. Total API cost to reproduce: **$0.44**. We verified reproducibility by running the full 3-seed experiment twice; per-seed accuracy varied by ≤ 0.5pp (1–2 queries out of 400), attributable to rate-limited API retries, with all qualitative conclusions unchanged.

References

Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*.

Larson, S., Mahendran, A., Peper, J. J., Clarke, C., Lee, A., Hill, P., Kummerfeld, J. K., Leach, K., Laurenzano, M. A., Tang, L., & Mars, J. (2019). An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. *Proceedings of EMNLP-IJCNLP 2019*, 1311–1316.

Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP 2019*.

Tunstall, L., Reimers, N., Jo, U. E. S., Bates, L., Korat, D., Wasserblat, M., & Pereg, O. (2022). Efficient Few-Shot Learning Without Prompts. *arXiv preprint arXiv:2209.11055*.

Anthropic. (2025). Claude Haiku 4.5 — Model Card and Pricing. docs.anthropic.com/en/docs/about-claude/models. Accessed 2026-04-05.